

AEL-0

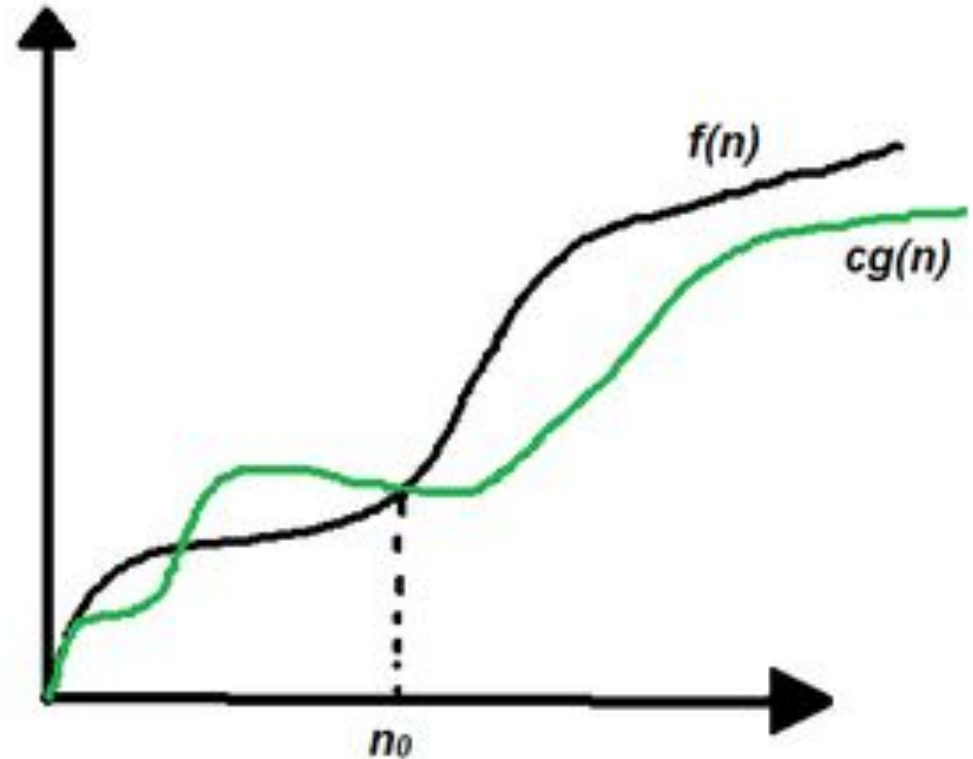
Asymptotic notation

Omega Notation (Best Case)

$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}.$

This means that, $f(n) = \Omega(g(n))$, If there are positive constants n_0 and c such that, to the right of n_0 the $f(n)$ always lies on or above $c \cdot g(n)$.

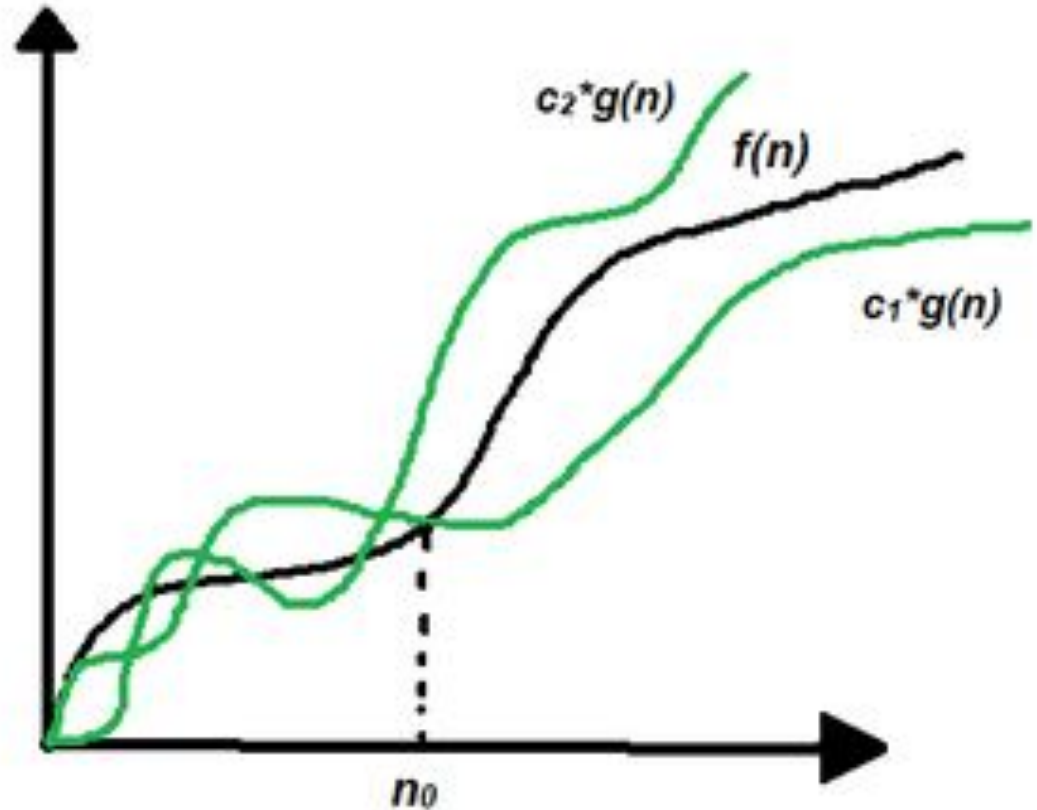
$g(n)$ is the lower bound for $f(n)$.



Theta Notation (Average Case)

$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

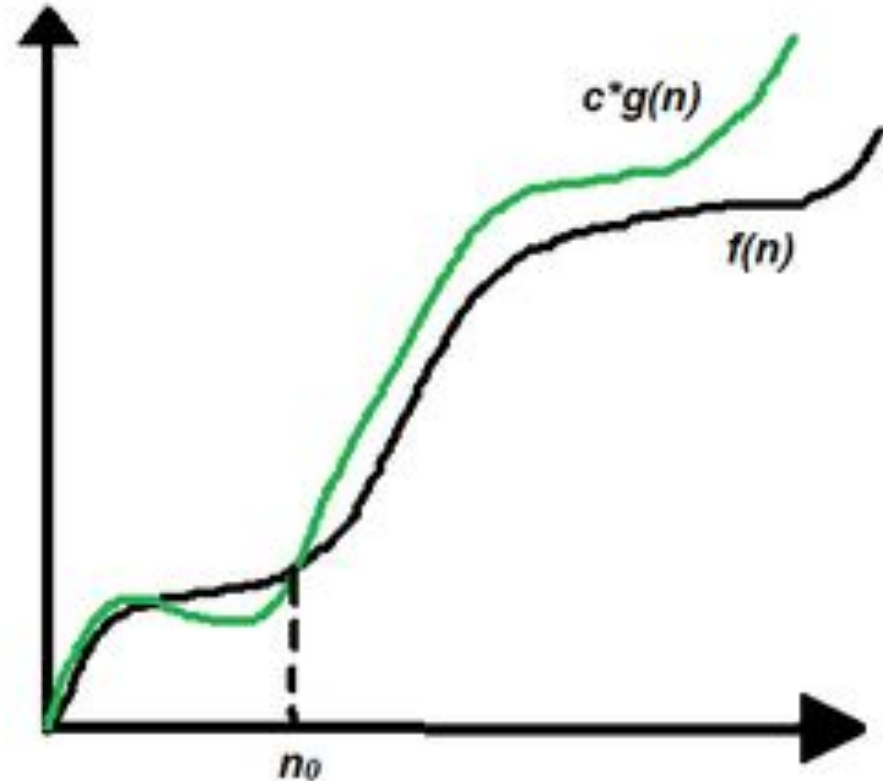
This means that, $f(n) = \Theta(g(n))$, If there are positive constants n_0 and c such that, to the right of n_0 the $f(n)$ always lies on or above $c_1 \cdot g(n)$ and below $c_2 \cdot g(n)$.



Big-O Notation (Worst Case)

$O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

This means that, $f(n) = O(g(n))$, If there are positive constants n_0 and c such that, to the right of n_0 the $f(n)$ always lies on or below $c \cdot g(n)$.



Example

INSERTION-SORT(A)

1 **for** $j = 2$ **to** $A.length$

2 $key = A[j]$

3 // Insert $A[j]$ into the sorted
 sequence $A[1..j-1]$.

4 $i = j - 1$

5 **while** $i > 0$ and $A[i] > key$

6 $A[i+1] = A[i]$

7 $i = i - 1$

8 $A[i+1] = key$

cost

times

c_1

n

c_2

$n - 1$

0

$n - 1$

c_4

$n - 1$

c_5

$\sum_{j=2}^n t_j$

c_6

$\sum_{j=2}^n (t_j - 1)$

c_7

$\sum_{j=2}^n (t_j - 1)$

c_8

$n - 1$

Continued

t_j means number of times inner while loop will run for the value j in the outer for loop.

The running time of insertion sort is,

$$\begin{aligned} T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) . \end{aligned}$$

Continued

Best Case :

- Array is already sorted, t_j is 1.
- The running time of insertion sort becomes a linear function.

Average Case :

- On average, half the elements in $A[0..j-1]$ are less than $A[j]$, and half the elements are greater. Therefore, we check half of the subarray $A[0..j-1]$, and so t_j is about $j/2$.

Worst Case :

- Array is sorted in reverse order.
- t_j is j .

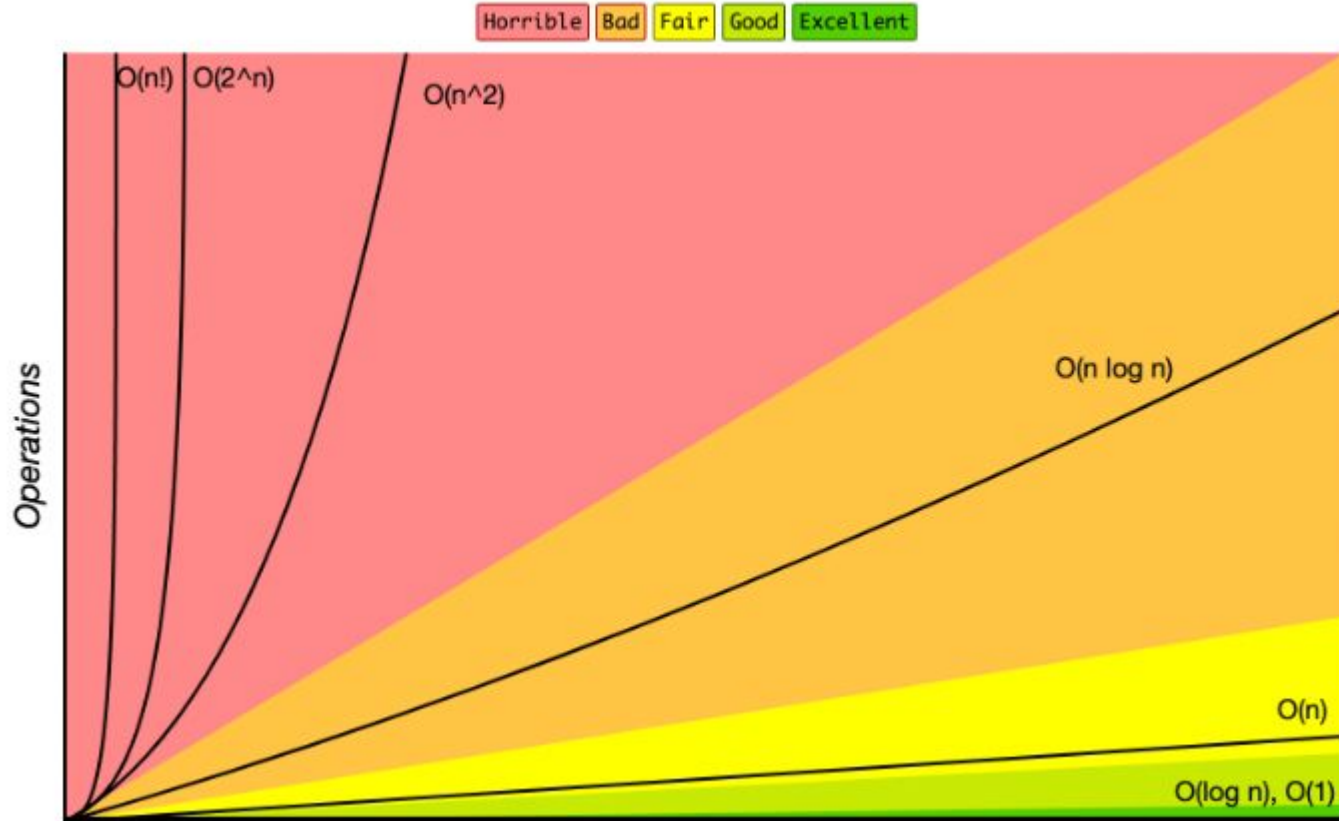
$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

and

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

Cheat Sheet

- $1 < \log(n) < n < n \log(n) < n^c < c^n < n!$ where n is input size and c is positive constant.



Continued

- When your calculation is not dependent on the input size, it is a constant time complexity ($O(1)$).
- When the input size is reduced by half, maybe when iterating, handling recursion, or whatsoever, it is a logarithmic time complexity ($O(\log n)$).
- When you have a single loop within your algorithm, it is linear time complexity ($O(n)$).
- When you have nested loops within your algorithm, meaning a loop in a loop, it is quadratic time complexity ($O(n^2)$).
- When the growth rate doubles with each addition to the input, it is exponential time complexity ($O(2^n)$).

Peak Finder 1D

Classic

- Go through every element and check if it is a peak or not.
- Cost $O(1)$ for best case when array is like $[9,8,7,6,5,4,3]$
- Cost $O(n)$ for both average case and worst case when array are like $[1,2,3,4,3,2,1]$ and $[1,2,3,4,5,6]$ respectively.

Divide and Conquer

- If $a[n/2] < a[n/2-1]$ look $a[1..n/2]$
- Elif $a[n/2] < a[n/2+1]$ look $a[n/2..n]$
- Else return $a[n/2]$
- Cost only $\text{Log}_2(n)$